



Universidade do Minho
Licenciatura em Engenharia Informática

Unidade Curricular de
Programação Orientada a Objetos
17 de Maio 2025

Trabalho Prático de Programação Orientada a Objetos

SpotifyUM

Grupo 35:

- ❖ Bruno Campos, nº98639;
- ❖ Tiago Brito, nº106796;

Índice

Índice.....	1
Índice de Figuras.....	2
1. Introdução.....	3
2. Arquitetura de Classes.....	4
2.1. Model.....	4
2.1.1. User.....	4
2.1.2. UserPremium.....	4
2.1.3. SubscriptionPlan.....	4
2.1.4. FreePlan, PremiumBase e PremiumTop.....	5
2.1.5. Music.....	5
2.1.6. Interpreter.....	5
2.1.7. Playlist.....	6
2.1.8. SpotifyUM.....	6
2.2. Controller.....	6
2.3. View.....	7
2.4. Diagrama de Classes.....	7
3. Funcionalidades.....	8
4. Conclusão.....	10

Índice de Figuras

Figura 1: Diagrama de Classes.....	7
Figura 2: Menu Principal.....	8
Figura 3: Menu de Estatísticas.....	8
Figura 4: Menu de Músicas.....	9
Figura 5: Menu de utilizadores.....	9
Figura 6: Menu da conta do utilizador “Tiago”	9

1. Introdução

Este relatório descreve o desenvolvimento do projeto SpotifyUM, realizado no âmbito da unidade curricular de Programação Orientada a Objetos (POO). O projeto cria uma aplicação para gestão de músicas, playlists e utilizadores, respeitando os princípios da programação orientada a objetos.

A arquitetura adotada baseia-se no padrão **MVC**:

- **Model**: Contém a lógica de dados e entidades.
- **View**: Responsável pela interação com o utilizador através de menus.
- **Controller**: Liga o utilizador ao sistema, tratando os pedidos e atualizando o modelo.

2. Arquitetura de Classes

2.1. Model

2.1.1. User

Representa um utilizador com atributos como:

- **name, email, password, morada e points;**
- **List<Playlist> userPlaylists;**
- **SubscriptionPlan subscriptionPlan.**

Esta classe representa um utilizador com o plano de subscrição **Free**, sendo posteriormente estendida pela classe UserPremium.

2.1.2. UserPremium

Tal como foi dito é uma extensão da classe User que suporta os planos pagos e com isso herda vários benefícios.

A decisão de fazer com que a classe UserPremium estenda a classe User foi tomada para facilitar a distinção entre os dois tipos de utilizadores e permitir uma modificação mais simples das funcionalidades específicas do utilizador premium. Desta forma, é possível reutilizar os métodos comuns a ambos os tipos de utilizadores na classe base (User), ao mesmo tempo que se especializa o comportamento do utilizador premium.

2.1.3. SubscriptionPlan

Interface implementada por todos os planos de subscrição, ditando que cada plano de subscrição inerente ao *SubscriptionPlan*, implemente os métodos para ser possível adquirir os pontos adicionais que o utilizador deverá ganhar quando ouve uma música, tal como o método para obter o multiplicador dos pontos de um utilizador com um certo plano.

Foi utilizada uma interface para garantir que todos os planos de subscrição têm uma implementação que permita não causar problemas na necessidade de calcular os pontos a ganhar quando o utilizador ouve uma música.

2.1.4. FreePlan, PremiumBase e PremiumTop

Implementações concretas da interface *SubscriptionPlan* com diferentes lógicas de pontuação:

- **FreePlan**: 5 pontos por música;
- **PremiumBase**: 10 pontos e multiplicador a x1;
- **PremiumTop**: 10 pontos e multiplicador a x1.025;

2.1.5. Music

É a classe que representa uma música com todos os seus atributos:

- **Music_counter**: contador de músicas criadas, para permitir obter Ids únicos para cada música
- **Id**: identificador único de uma música
- **Nome**: nome da música
- **Interpreter**: nome do interpretador da música
- **Editor**: editora da música
- **Lyrics**: letras da música
- **Genre**: gênero musical da música
- **Duration**: duração da música em segundos
- **Listeners**: número de vezes que a música foi ouvida
- **HasVideoClip**: booleano que verifica se a música têm um videoclipe associado
- **isExplicit**: booleano que verifica se a música é explícita

2.1.6. Interpreter

Classe que representa um interpretador de uma música, a decisão para a criação da classe Interpreter foi para tornar mais fácil a aquisição das estatísticas do artista.

- **Listeners**: número de vezes que o interpretador foi ouvida
- **Name**: nome do interpretador

2.1.7. Playlist

Classe que representa uma playlist, ou seja, uma coleção de músicas:

- **List<Music> musics:** lista de músicas presentes na playlist
- **publica:** booleano que verifica se a playlist é pública
- **nome:** nome da playlist

2.1.8. SpotifyUM

Classe principal que guarda e mantém todas as classes no model.

- **Map<Integer, Music> musics:** Todas as músicas existentes na aplicação
- **Map<String, User> users:** Todos os users existentes na aplicação
- **Map<String, Interpreter> interpreters:** Todos os intérpretes existentes na aplicação

2.2. Controller

Em todos os controllers vão existir os mesmos elementos, sendo estes:

- **SpotifyUM:** Classe que mantém a lógica de negócio da aplicação e guardas os valores das classes em Model;
- **View:** Interface que o utilizador vê;
- **Scanner:** Lê o input do utilizador.

Sendo assim, temos os seguintes ficheiros:

- **UserController:** Gere a autenticação, registo, e outras operações sob o domínio do utilizador;
- **MusicController:** Gere a inserção e listagem de músicas;
- **StatisticalController:** Permite obter várias estatísticas em relação ao programa;
- **Controller:** É a classe principal que controla a aplicação, pois instrui os outros controladores e realiza o ciclo principal de execução.

2.3. View

Existem 3 *views* distintas que têm funções diferentes, porém tudo a ver com o menu mostrado.

- **Text:** Mantém variáveis que permitem a mudança da cor do texto apresentado ao utilizador;
- **Menu:** Gera os menus dinamicamente, onde é possível ao utilizador interagir com o *controller*;
- **View:** Classe que mostra resultados ao utilizador, pede *inputs* e mostra mensagens de erro ou de confirmação.

2.4. Diagrama de Classes

No fim disto tudo, chegamos ao seguinte diagrama de classes:

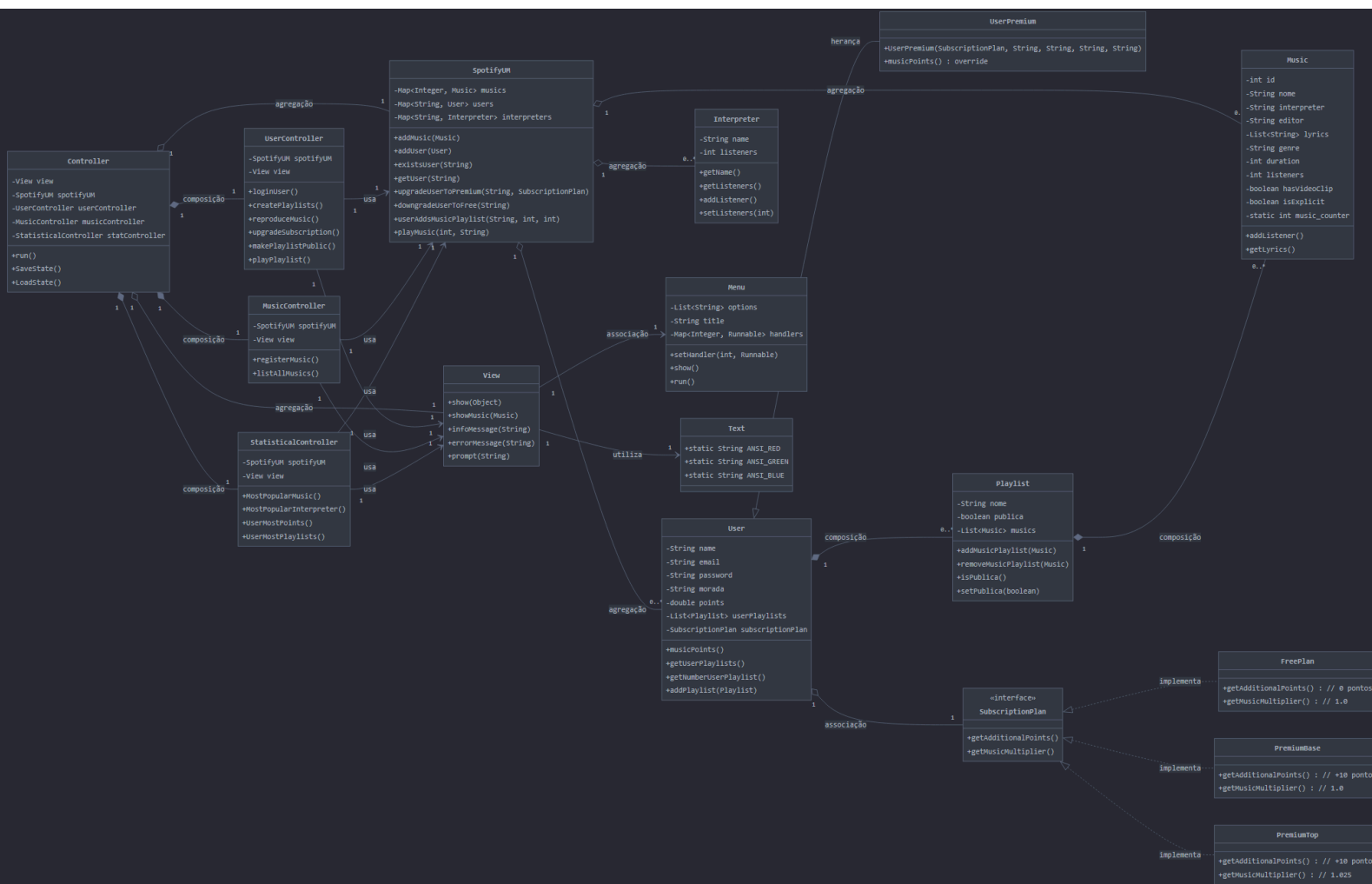


Figura 1: Diagrama de Classes

3. Funcionalidades

De seguida são mostrados resumidamente os menus da aplicação e as opções implementadas.

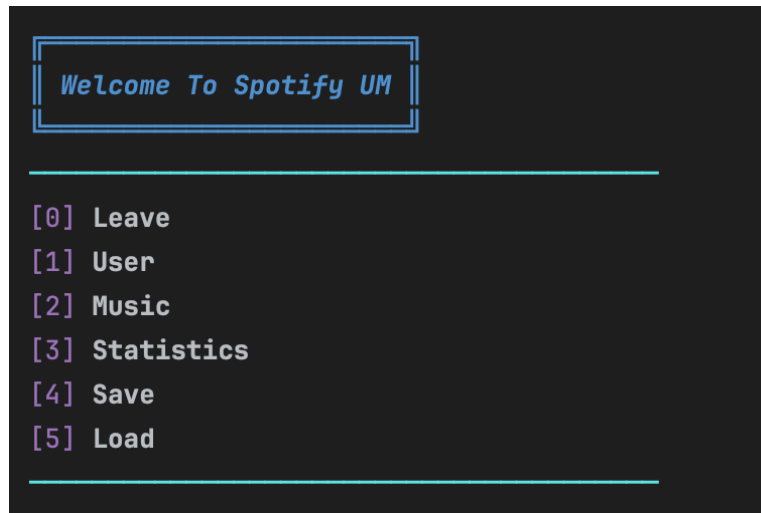


Figura 2: Menu Principal

Quando a aplicação é inicializada é apresentado o menu acima. A opção '0' desliga a aplicação, a opção '4' salva o estado da aplicação, havendo persistência de dados devido ao método 'save(SpotifyUM sp)' da classe Controller que grava o objeto num ficheiro binário, sendo todas as classes do *Model* guardadas nesse ficheiro, e a opção '5' da load do estado salvaguardado.

As opções '1', '2' e '3' levam aos controllers UserController, MusicController e StatisticsController, respetivamente.

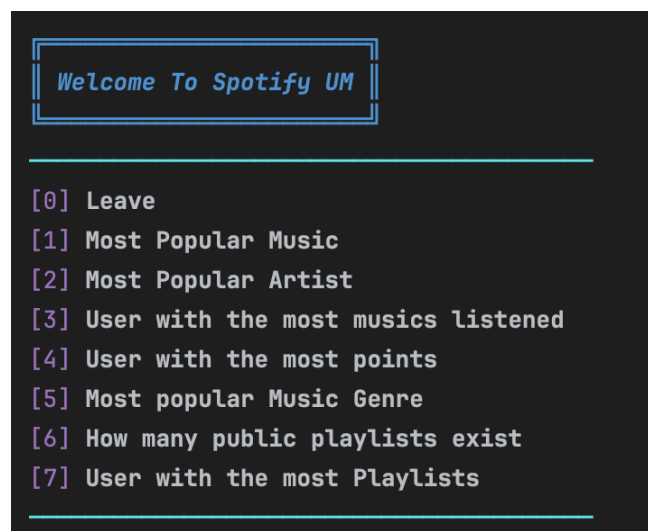


Figura 3: Menu de Estatísticas

No menu das estatísticas é possível verificar as diferentes opções implementadas sobre a obtenção de estatísticas sobre o programa.

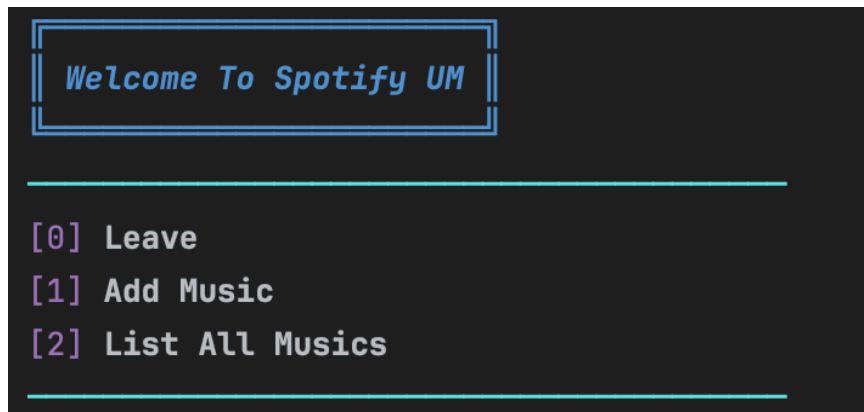


Figura 4: Menu de Músicas

No menu músicas é possível adicionar novas músicas e obter uma lista de todas as músicas presentes na aplicação.

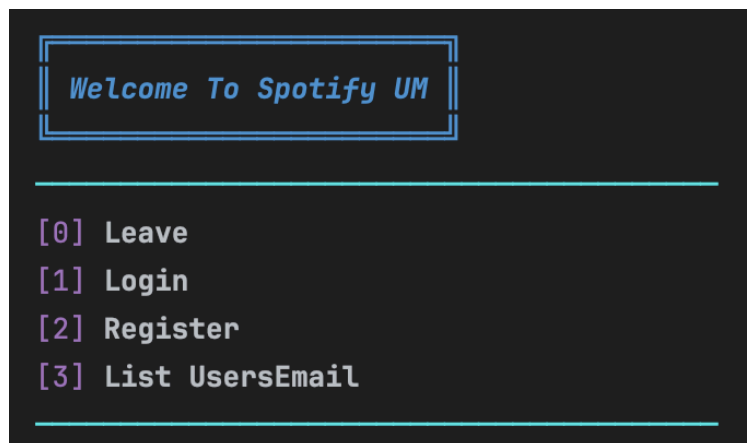


Figura 5: Menu de utilizadores

No menu dos utilizadores é possível ver todos os utilizadores pelo seu e-mail, aparecendo o seu nome e email, registar um utilizador e entrar na conta de um utilizador já registado.

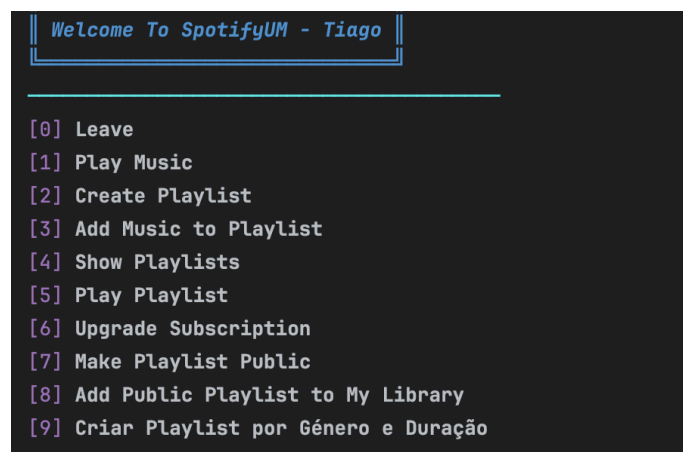


Figura 6: Menu da conta do utilizador "Tiago"

Fazendo o login com o email e password, o utilizador conseguirá ver o seguinte menu onde é possível ver as diversas opções permitidas pela aplicação.

4. Conclusão

O desenvolvimento do projeto SpotifyUM permitiu consolidar os principais conceitos de Programação Orientada a Objetos, aplicando de forma prática o padrão MVC, com clara separação entre modelo de dados, controlo da lógica e interface com o utilizador.

Através do uso de interfaces, herança e composição, foi possível criar uma estrutura modular e de fácil manutenção. As funcionalidades implementadas cobrem não só os requisitos essenciais (como gestão de utilizadores, músicas e playlists), mas também funcionalidades avançadas como reprodução com controlo por tipo de plano, estatísticas em tempo real, e persistência de dados via serialização.

A arquitetura criada permite escalar facilmente a aplicação no futuro, com adição de novos planos de subscrição, tipos de playlists ou interfaces gráficas.

Em suma, este projeto revelou-se uma excelente oportunidade para aplicar os conhecimentos adquiridos ao longo das aulas e desenvolver um sistema coeso, funcional e alinhado com os princípios da programação orientada a objetos.